

Searching Search Spaces: Meta-evolving a Geometric Encoding for Neural Networks

Tarek Kunze
ISAE-SUPAERO,
Université de Toulouse,
Toulouse, France
tarek.kunze@protonmail.com

Paul Templier
ISAE-SUPAERO,
Université de Toulouse,
Toulouse, France
paul.templier@isae.fr

Dennis G. Wilson
ISAE-SUPAERO,
Université de Toulouse,
Toulouse, France
dennis.wilson@isae.fr

Abstract—In evolutionary policy search, neural networks are usually represented using a direct mapping: each gene encodes one network weight. Indirect encoding methods, where each gene can encode for multiple weights, shorten the genome to reduce the dimensions of the search space and better exploit permutations and symmetries. The Geometric Encoding for Neural network Evolution (GENE) introduced an indirect encoding where the weight of a connection is computed as the (pseudo-)distance between the two linked neurons, leading to a genome size growing linearly with the number of genes instead of quadratically in direct encoding. However GENE still relies on hand-crafted distance functions with no prior optimization. Here we show that better performing distance functions can be found for GENE using Cartesian Genetic Programming (CGP) in a meta-evolution approach, hence optimizing the encoding to create a search space that is easier to exploit. We show that GENE with a learned function can outperform both direct encoding and the hand-crafted distances, generalizing on unseen problems, and we study how the encoding impacts neural network properties.

Index Terms—evolution strategies, genetic programming, meta-evolution, encoding, neural networks, reinforcement learning, policy search

I. INTRODUCTION

Artificial Neural Networks (ANNs) are at the heart of modern deep learning systems, behind applications in computer vision, control, and language processing [1]. The Universal Approximation Theorem [2] posits that any function can be approximated by a neural network, but efficiently finding the architecture and parameters that define the neural network approximating a function has been the focus of research for the last three decades.

In many cases where gradients are hard to obtain, Evolutionary Algorithms (EA) have been used to optimize neural networks. Policy search, e.g. finding the network that represents the best decision function (“policy”) for an agent interacting with an environment, presents a situation where the optimal policy is not known in advance hence the exact error cannot be computed. EAs have been successfully applied to policy search, yielding results competitive with gradient-based Reinforcement Learning (RL) approaches [3]. Using an EA to optimize the parameters of an ANN requires representing the network as a genome: the most straightforward approach is to concatenate its parameters in one vector. In this “direct” encoding each gene represents a single parameter. As most neural network

architectures rely on fully-connected layers where each neuron from a layer takes as inputs the activation values of all the neurons from the previous layer, the number of weights in one of these layers grows quadratically with the number of neurons and so does the size of the genome which in turns increases the optimization cost [4].

To reduce the genome size scaling to linear with the number of neurons, the Geometric Encoding for Neural network Evolution (GENE) [4] framework introduced an indirect encoding method where the weight of each connection is generated by computing the (pseudo-)distance between the coordinates of the two connected neurons in a latent space. The neuron coordinates are then optimized with the EA, each coordinate being used to generate the weights of all incoming and outgoing connections for that neuron. Smaller genomes reduced the optimization time and memory costs by 2 orders of magnitude with the Exponential Natural Evolution Strategy (XNES) [5] on Atari control tasks. However GENE relies on hand-crafted distance functions and is still outperformed by direct encoding on many tasks, showing the subspace of neural networks attainable through the encoding is either harder to search or does not allow to represent best-performing policies that direct encoding would allow. In this work we propose to optimize the distance function with a meta-evolution process, using Cartesian Genetic Programming (CGP) [6] to represent it as an explainable graph. We show that CGP can find a simple GENE encoding function that outperforms the hand-crafted method and direct encoding, generalizing to control problems not seen during training.

II. RELATED WORKS

Optimizing a neural network usually combines two parts: defining its architecture by choosing which neurons are connected, and fixing the value of the weight on each connection. These steps can be done at the same time with methods such as Neuroevolution of Augmenting Topologies (NEAT) [7] which builds the network by progressively adding connections, or separately where an outer loop searches for the architecture while an inner method optimizes the parameters of a fixed architecture. While the architecture optimization step is studied by the field of Neural Architecture Search [8], we focus on the task of finding the optimal parameters for a fixed architecture.

A. Encoding neural networks

Using evolution as an optimization methods for deep neural networks requires the parameters of the network to be encoded as a genome. Direct encoding concatenates all the weights and biases of a neural network into one vector $\sigma \in \mathbb{R}^D$, the genome. The size of the genome grows linearly with the number of connections in the network. However when connecting two consecutive layers of size a and b , there are $a \times b$ weights to consider. The size of the genome then grows quadratically with the number of neurons in each layer.

Inspired by the biological genotype-to-phenotype mapping where one gene can impact the development of many parts of the body, indirect encoding methods aim to change the search space by adding a transformation between the genome and the neural network it defines. This transformation can reduce the dimensions of the genome reducing optimization costs, but also change the typology of the search space to help find high-performing solutions as neural networks show many permutations and symmetries which could be exploited. As in this work we focus specifically on the GENE indirect encoding [4] (see section II-C), we refer to their paper for a more detailed review of indirect encoding methods.

B. Optimizing neural networks

Once the encoding of policy networks into genomes as vectors of continuous values is defined, policies can be optimized as black-box problems with evolutionary methods such as genetic algorithms [9] or evolution strategies (ES) [10]. The evaluation of a genome is done by building the network with the weights from the genome, and using the network as a policy in an episode of the problem. The total reward over the episode is then used as fitness for the evolutionary algorithm. Optimizing the parameters of the genome is hence equivalent to optimizing the policy the network represents. Usually used for the continuous optimization of parameters in a neural network, Evolution Strategies are a subset of evolutionary algorithms that rely on a distribution in the genome space to sample solutions to evaluate. The distribution is then updated based on the ranking of solutions to favor the best performing ones, iteratively until convergence. ES have been shown to be competitive with deep reinforcement learning methods on robotics and image-based control tasks [3].

In the evolution of neural networks, the ES is not aware of the encoding used since the process is black-box. The encoding however changes the search space by potentially reducing its dimensionality and how it can be navigated. While characterizing the optimization landscape is outside the scope of this work, the change in genome size can have a direct impact on optimization cost. This is especially the case with ES methods that compute the covariance matrix of the distribution like Covariance Matrix Adaptation Evolution Strategy (CMAES) [11] or Exponential Natural Evolution Strategy (XNES) [5]. As their compute cost grows quadratically with the size of the genome, their use is still limited to small policies. Variants of these methods like Separable CMAES (SepCMA) [12] or Separable Natural Evolution Strategy (SNES) [5] have been

	HALFCHEETAH	WALKER2D	HOPPER	SWIMMER
Direct	19718	19590	18435	17922
GENE	1102	1099	1069	1056
Ratio	18	18	17	17

Table 1: Genome size for GENE ($d = 3$) and direct encoding with the same neural network architecture used here, and the ratio between.

introduced to tackle the cost by considering all optimization dimensions as separable, which reduces the compute cost but impacts performance.

C. A Geometric Encoding

The Geometric Encoding for Neural Network Evolution (GENE) [4] projects all the neurons of a fully-connected neural network into a latent Euclidean space of small dimensions d , then optimizes the concatenated coordinates of these neurons using Evolution Strategies. To generate the network used as a policy, GENE then computes the weight of each connection as the distance between the two connected neurons in the latent space. As the genome for N neurons is only dN parameters, the GENE encoding reduces the size of the genome by an order of magnitude for the same network architecture (see Table 1). With ES methods like CMAES where the memory cost grows quadratically with the genome size, an 18-fold reduction of the genome means reducing the size of the covariance matrix in memory by 324 times, making large networks tractable.

The “distance” term is here used loosely to describe a function that takes two points in a Euclidean space and returns a value in $\mathbb{R}$. A proper distance function in the mathematical sense would only create positive values, but not having negative weights in the network would not allow to have inhibition behaviors where the activation of a neuron reduces the activation of another. GENE is defined with the pL2 distance function (2) based on the Euclidean distance with an additional factor allowing for negative values (1).

$$\alpha : \begin{cases} \text{if } x \geq 1 : \alpha(x) = 1 \\ \text{if } x \leq -1 : \alpha(x) = -1 \\ \text{else: } \alpha(x) = x \end{cases} \quad (1)$$

$$d_{pL2}(n_1, n_2) = \alpha\left(\prod_{i=1}^D n_1^i - n_2^i\right) \sqrt{\sum_{j=1}^D (n_1^j - n_2^j)^2} \quad (2)$$

$d_{pL2}(n_1, n_2)$ is then used as the weight of the connection from neuron n_1 to neuron n_2 , and the ES optimizes the values of n_1^i and n_2^i for all latent dimensions i . Instead of hand-crafting distance functions like pL2 with human bias, we here propose to automatically discover them with meta-evolution.

D. Meta-evolution

While usually manually defined through human engineering, algorithms can be automatically improved with the approach of meta-learning. Meta-learning methods have been applied to a variety of problems with the same underlying goal of replacing

hand-crafted portions of a learning system with automatically discovered settings. This can be hyperparameters, as in Neural Architecture Search (NAS) and Hyperparameter Optimization, or entire portions of a learning algorithm. Learning to learn effectively is what drives the use of meta-learning methods. Evolutionary methods have a long history of being used in designing programs due to their derivative-free properties allowing them to work effectively on problems with unknown functional form [13]. Their population-based nature allows them to leverage the latest hardware advances, making them ideal for hardware accelerators that utilize parallel operations. Meta-learning works by turning parts of a learning algorithm into optimizable sections which are then learned by an outerloop.

Deep Reinforcement Learning methods have been a target for meta-learning to either automatically rediscover and improve on existing methods by representing algorithmic components as graphs, which can be optimized with Genetic Programming [14]. Meta-evolved functions can also be represented with neural networks optimized by an Evolution Strategy [15], with additional analysis being required to transform the learned network into a function that can easily be implemented as a standard learning algorithm. Gradient-free evolutionary meta-evolution methods [16] outperform gradient-based ones [17] by allowing the meta-evolution to account for the complete training process of the optimized algorithm in one fitness value.

Meta-learning parts of evolutionary optimisation algorithms was also studied with Learned Evolution Strategies (LES) [18] and Learned Genetic Algorithms (LGA) [19] where a meta-learning loop with a self-attention based framework searches over the learning rates and weights assignment rules of an ES, or over the selection and mutation rate adaptations of a GA. They create fine tuned update rules which outperform previous human-engineered ES and GA respectively.

E. Cartesian Genetic Programming

As seen in previous work on the meta-evolution of learning methods (see section II-D), the function to optimize can be represented by a neural network but the learned output then needs additional processing to be implemented as a standard learning method or studied analytically. Cartesian Genetic Programming (CGP) [6] instead represents programs as graphs with discrete functions linked together, which can then be translated into interpretable code. CGP indexes the nodes of the GP graph with Cartesian coordinates and has been successfully applied to a range of problems from boolean functions [6] to policy search in image-based Atari environments [20].

III. METHOD

The pL2 distance function introduced in the GENE article has a number of shortcomings, including the impact of any mutation on both incoming and outgoing connections which can lead to a loss of expressiveness by the target network. However finding a better distance function is a complex process if done manually. Instead of defining a new function by hand, we use meta-evolution to learn a new one that performs well on

a set of defined training tasks and evaluation criteria. The meta-evolution loop is defined in Fig. 1.

A. Distance function optimization

Distance functions are represented and optimized using Cartesian Genetic Programming: the loop generates a population of candidate distance functions that are evaluated on a set of defined metrics then combined into the final fitness, which is used to update the state of the GP method.

A GENE distance function, such as L2 or pL2, can be described by an acyclic directed graph taking as input the position vectors of the input neurons, e.g. their coordinates in the latent space introduced by the encoding. If the dimension of the latent space is $d = 3$ then the input vector has size $2d = 6$. To allow for constants, CGP extends the input vector with additional nodes representing possible constants (see Appendix A). The CGP graph has a single output, which is the weight of the connection in the network. The resulting graph can either be used in its genome form, which is what we do during the meta-evolution process, or in its symbolic form after selection and extraction of a function and then used as is in the code thanks to the explainability of Genetic Programming.

The main meta-evolution loop is simply a standard Cartesian Genetic Programming algorithm optimizing the distance functions. Once the distance functions have been generated, they will be evaluated according to three criteria: training performance, generated network properties and number of used inputs in the CGP distance function graph.

B. Training performance

Each distance is first evaluated on its ability to allow an ES to optimise neural networks as policies for control tasks over a full optimization run. During the evaluation phase, the distance functions are used to train policies on a set of control tasks $\mathcal{T}_{\text{train}}$ separately and the fitness of the best individual found is kept. Final maximum fitness values then goes through min-max normalization to have comparable scales and are summed to give a f_{task} score:

$$f_{\text{task}} = \sum_{t \in \mathcal{T}_{\text{train}}} f_t$$

We use more than one environments for the training phase of the meta-evolution to avoid overfitting the encoding on one problem. While this is expensive, the meta-evolution loop is only needed once to find a good distance function which can then be applied to any problem.

C. Evaluating network properties

The L2 distance function first tested for GENE is a good example of ill-formed distance function as it can only create positive values, removing the possibility for inhibition behavior to emerge in the network. To bias the meta-evolution search towards functions that can create usable neural networks we introduce regularization terms in the CGP fitness, described below.

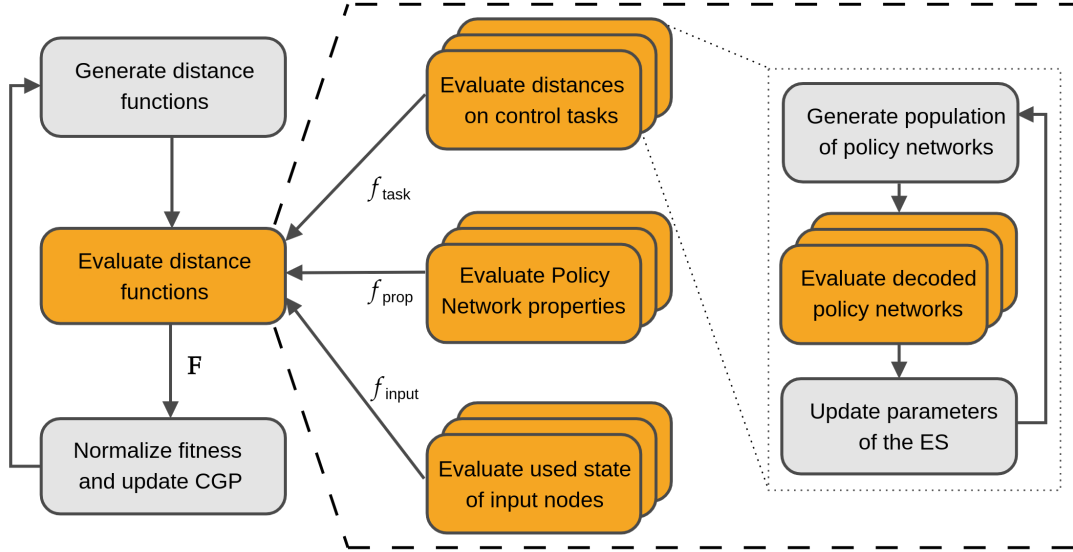


Fig. 1: Meta-evolution process, depicting the different steps happening. In orange are the evaluation steps conducted in parallel. **Left:** The outer loop is a CGP process, generating distance functions and evaluating them. **Center:** The components of distance function evaluation. The final fitness used for the CGP outer-loop F is put together and propagated to the update step of the meta-loop. **Right:** The distance function being evaluated is used as GENE encoding in a loop to optimize neural networks with and ES. This operation is done on both HALFCHEETAH and WALKER2D with f_{task} the sum of normalized final fitness scores.

1) *Weight distribution:* Neural networks trained with direct encoding show a weight distribution following a Gaussian distribution centered around 0 (see Fig. 3), which is consistent with standard ANN initialization methods as it allows for a balanced number of activator and inhibitor connections [21]. We favor distance functions which will create networks with this property by sampling random genomes from a Gaussian distribution and measuring the distribution of the weights. We note the value of the mean of the weights as f_{mean} as well as the square of the difference between the empirical standard deviation and the target σ_f as f_{std} . These penalties encourage networks whose weight distribution is that of a Gaussian centered at μ_f and with a standard deviation of σ_f .

2) *Input distribution restoration:* Neural networks trained on data should be able to pick out the underlying distribution of the data. In order to do so, the initial weight distribution should ideally be symmetric [21]. This property is tested by running samples throughout the network and evaluating the obtained distributions mean with respect to the input one. The absolute difference from the mean is denoted as f_{sym} .

D. Using all inputs

CGP has a bias towards functions which do not use all inputs due to the graph representation and mutation operator. To push the evolved distance function to use all the dimensions of the projected space, we introduce a bonus term f_{input} scaled by an hyperparameter α (see Fig. 1) which is maximized when all 6 input nodes of the graph are used. Unused dimensions in a learned distance functions will hence highlight that no evolutionary advantage was found by increasing the graph complexity.

E. Weighted integration

To calculate the final fitness used for a distance function, we first sum the network weight distribution terms to make f_{prop} :

$$f_{\text{prop}} = f_{\text{mean}} + f_{\text{std}} + f_{\text{sym}}$$

This term expresses how far the evolved network weights are from the desired distribution, i.e. that the network weights are distributed normally around 0 with a small standard deviation and represent a symmetric function. As this term is expressed as a penalty, the term f_{prop} is subtracted from the task fitness f_{task} in the final fitness F :

$$F = \beta f_{\text{task}} - (1 - \beta) f_{\text{prop}} + \alpha f_{\text{input}}$$

The fitness F is therefore the sum of both task fitness values and the bonus for using all inputs f_{input} , minus the penalty of network properties. We used a β of $1/3$, meaning that greater focus was given to the network properties than the task fitness.

IV. EXPERIMENTS AND RESULTS

A. Experimental details

We first meta-learn a collection of distance functions on 2 tasks: $\mathcal{T}_{\text{train}} = \{\text{HALFCHEETAH}, \text{WALKER2D}\}$ and we set $\mu_f = 0$ and $\sigma_f = 0.5$ for f_{mean} . The meta-loop was started for 5000 generations, using $\beta = 1/3$ and $\alpha = 4\beta$. All hyperparameters used are presented in the appendix, table B.

ID	Function	HALFCHEETAH	WALKER2D	HOPPER	SWIMMER
Direct		1561	2694	1468	297
pL2		1501	<u>2172</u>	1754	<u>298</u>
LD-10	x_2	7649	1194	232	214
LD-79	$\sin(\exp(z_1) * y_2)$	7793	1326	642	253
LD-204	$\sin((((z_2 - x_1) > z_2) + (z_2 - ((z_2 - x_1) > 0.1))) < \text{abs}(z_2)) * y_2)$	7669	1574	<u>2164</u>	286
LD-206	$\sin(\text{abs}(z_2 > x_1) * y_2)$	7853	1298	2040	193
LD-318	$\text{abs}(z_2 > x_1) * y_2$	7898	1334	1877	243
LD-352	$\sqrt{z_2 > x_1} * y_2$	<u>7898</u>	1334	1877	243
LD-367	$\sqrt{x_2 > z_1} * y_2$	7766	1944	2211	291
LD-376	$((x_2 > x_1)/1) * y_2$	7744	1650	1850	250
LD-573	$((1 * z_2) < ((x_2 - (1 * x_1)) - \cos(x_2))) < (0.1 * ((1 < \text{abs}(z_1)) < z_2)))$	7701	1196	629	167
LD-626	$\sin(x_2 < x_1) * y_2$	7910	1241	1046	233

Table 2: Top 10 distance functions found by the meta-evolution with corresponding average final scores on control tasks. Highest fitness in bold, second highest underlined. The best distance function LD-367 is used in the rest of the paper.

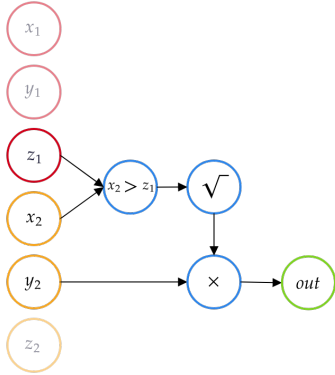


Fig. 2: Visualization of the CGP graph of the learned distance function. In **red** are the input coordinates of the first neuron and in **orange** are the input values of the second neuron. In **blue** are the operator nodes, used by CGP to construct a function. The output node is depicted in **green**. The function can be simplified as $d = (x_2 > z_1)y_2$, as $>$ is a boolean operator. In total, 3 input nodes, 3 operator nodes and the single output node are used.

B. Discovered GENE encodings

During the meta-evolution process, the distance functions that obtain the highest fitness are archived. From these archived distance functions, we evaluate the best $k = 10$ on a series of environments $\mathcal{T}_{\text{test}} = \mathcal{T}_{\text{train}} \cup \{\text{HOPPER, SWIMMER}\}$, including new environments not seen during training to assess their generalization capabilities. Functions and results are presented in Table 2. The ID corresponds to the generation in the meta-evolution where the distance appeared, with "LD" standing for "Learned Distance". We use LD-367 as learned distance in the other figures as it performs well on all environments. Functions are shown as learned by CGP, but they can be reduced as the square root or the absolute value of a boolean does not change its value. LD-318 and LD-352 are for example functionally identical and reach the same scores.

C. Best learned function

1) *Function graph*: We select LD-367 as the best distance function, reaching high fitness values on all environments, including ones not seen during the meta-evolution. Its graph is can be visualized in Fig. 2.

The function is very simple, using only 3 of the 6 input coordinates. With n_1^l and n_2^{l+1} two nodes connected by the weight $w_{1,2}^l$, their respective position vectors in the latent space are $n_1^l = [x_1, y_1, z_1]$ and $n_2^{l+1} = [x_2, y_2, z_2]$. The value of this specific weight is defined by the decoding operation in equation 3.

$$w_{1,2}^l = d_{367}(n_1^l, n_2^{l+1}) \quad (3)$$

$$= (x_2 > z_1)y_2 \quad (4)$$

As $>$ is a boolean operator we get $w_{1,2}^l \in \{0, y_2\}$, so this learned distance function acts a pruning operator, putting some weights to 0 and propagating y_2 to the others (connection-specific). Of the 64 available operator nodes that CGP could use to construct a distance function, only 3 were ultimately used. But these 3 nodes, in addition to GENE, were enough to simulate a pruning operator and ultimately beat a carefully hand-crafted distance function such as pL2 or tag-gene [4]. We show in Fig. 3 the distribution of weight values in neural networks trained with each encoding: the learned encoding

2) *Evaluating the encoding*: The learned encoding is then compared to direct encoding and to the hand-crafted original pL2 distance for GENE on control tasks $\mathcal{T}_{\text{test}}$. Training curves are presented in Fig. 4.

On 3 of the 4 environments, the learned distance functions performs better than pL2 and on 2 of them, HALFCHEETAH and HOPPER, it even beats direct encoding. On SWIMMER, the learned distance function takes longer to attain a good return value, with a very small gap between pL2 and direct encoding. By extrapolating the curve, we can imagine that the learned distance function has still plenty of room for improvement in comparison to pL2 and direct encoding who converge really fast but are stuck for the rest of the optimization.

Since Evolutionary Strategies have a high variance between individuals of the same population, and even if what is ultimately of interest is the best found individual, a more robust measure of the performance of an ES is to measure the

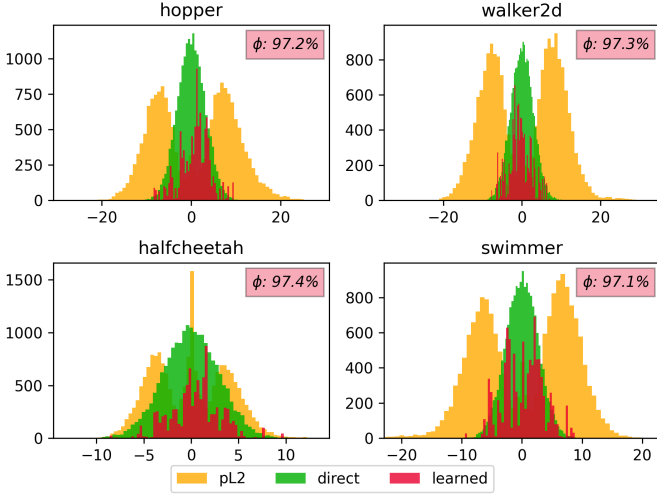


Fig. 3: Weight values distribution for neural networks trained with direct encoding and GENE with pL2 and learned distances. ϕ , is the percentage of weights at 0 using the learned distance function, not shown on the graphs.

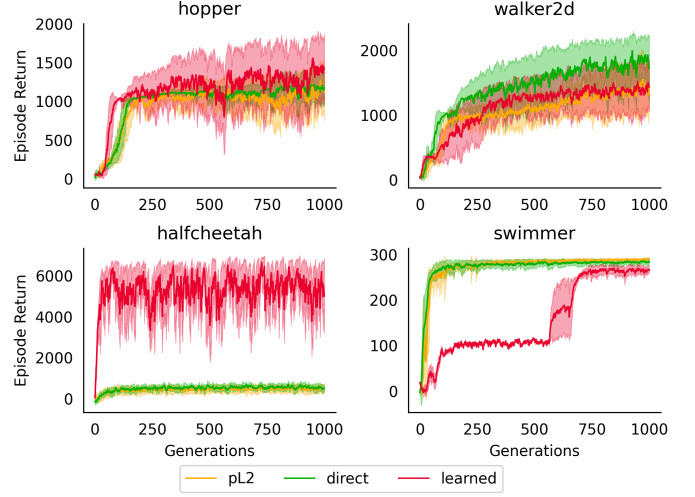


Fig. 5: Fitness of the center individual of the population, averaged over 5 run. The noisiness can be explained by the fact that the mean of the population is not very stable compared to the best observed individual.

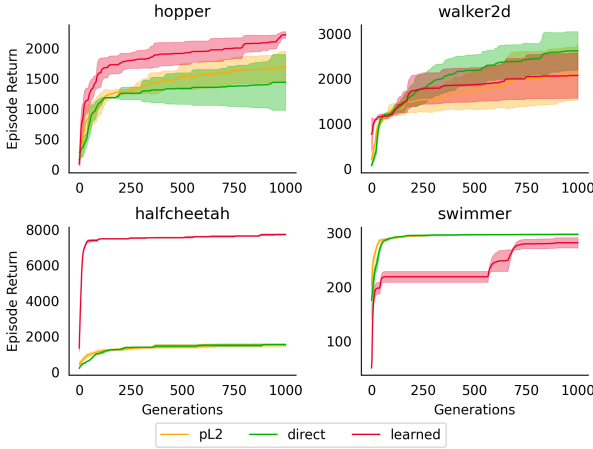


Fig. 4: Maximum fitness reached by an ES with each encoding, averaged over 5 runs.

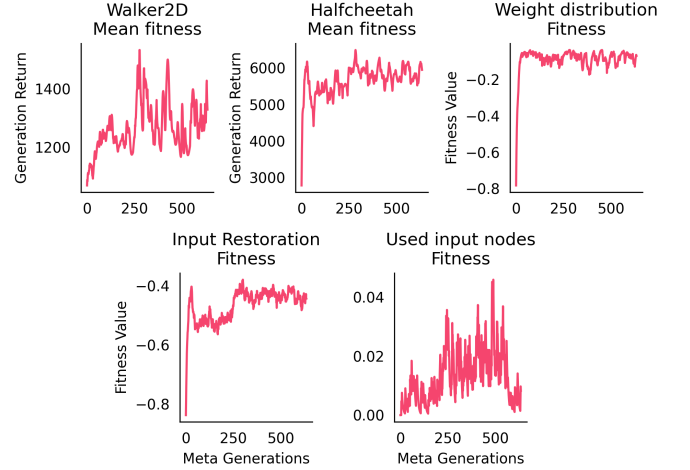


Fig. 6: Training curves of the Meta-evolution, for a few collected fitness values during training.

centre of the population. The same center used as a reference point to sample new candidates. Fig. 5 show us the learning curves obtained by evaluating the center of the population. The obtained results lead us to draw the same conclusion than those for the overall best individual.

D. Meta-evolution

Taking a step back from the final result, the whole meta-evolution process can be analysed in Fig. 6. The process ran for 635 generation and since no notable improvement can be observed after around 500 generations, we decided to stop the training. After quickly reaching 1200, the fitness value for WALKER2D oscillates between 1200 and 1400, while showing a slight upward trend. The training on HALFCHEETAH quickly

arrives at very good-performing policies using the learned distance functions.

As expected, the 2 enforced network properties (weight distribution and input restoration capabilities of the policy networks) show fast convergence. So the learned distance functions correctly enforced the defined network properties and used this to their advantage to find good distance functions. The used input nodes property, however, had no defining influence on the learning process.

The proportion of distance functions that use all 6 input values, i.e. all information in the coordinates of the projected neurons, is very low throughout training. This suggests that using all input values is not mandatory to find good performing distance functions in the current setting.

E. Analysing network properties and fitness

In order to observe the evolution of the characteristics of the political networks, we also evaluated these properties during the evaluation of the distance functions found, separating them by encoding type used.

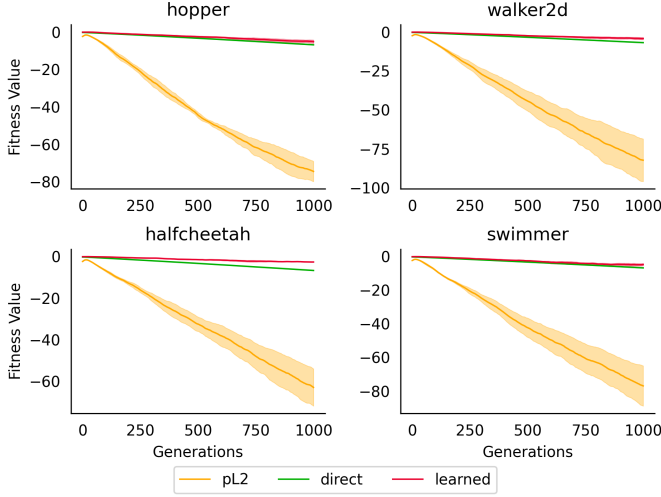


Fig. 7: Fitness of the weight distribution of the policy networks decoded with the learned distance function over the generations. Values are averaged during training over all policies and over 5 different runs.

We note in Fig. 7 that the distribution of the policy network weights encoded with the learned distance function (see 2), remains well centred around 0 and with a variance close to 0.5. This can be explained by the low penalty value obtained during training.

What is surprising in Fig. 7 is the high number of weights with the exact value of 0. When analyzing the evolved distance functions, we note that many of them include logic which results in exactly 0 for weights, i.e. the use of logical functions like $(x_2 > z_1)$. This results in sparse networks: networks which have a small percentage of non-zero weights. Sparse neural networks have been well-studied [22] and it is known that sparse versions of dense networks can be found through pruning [23], however it is surprising that evolution discovered a way to make sparse networks through the evolved distance functions, and that these distance functions generalize well to other tasks. This joins the analysis of [24], that high-performing sparse networks can be found by evolutionary strategies, while also proposing an encoding scheme to search only over sparse networks.

The importance of the input distribution restoration property is harder to analyze, see Fig. 8. It seems that it is entirely task-specific and that there are no links between the best found policy, and the networks ability to enforce this specific property. Indeed, the input distribution restoration property is only of interest as an initial starting point for an untrained neural network.

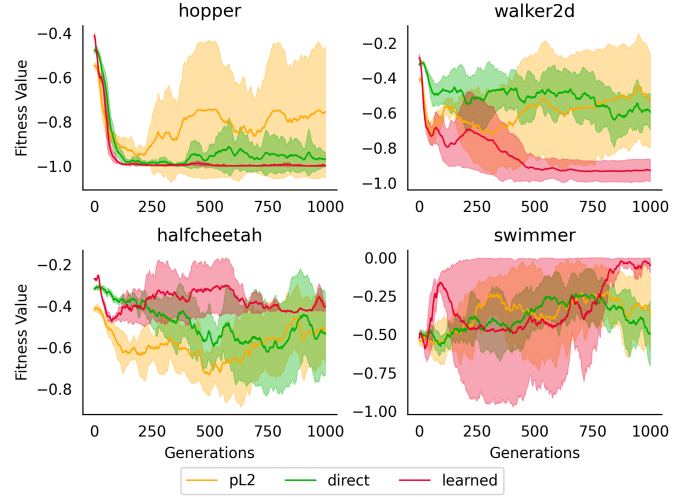


Fig. 8: Fitness of the input restoration ability of the policy networks decoded with the learned distance function over the generations. Values are averaged during training over all policies and over 5 different runs.

V. DISCUSSION

We showed that meta-evolving a distance function for the GENE encoding allows for better results on a set of continuous control benchmarks. Leveraging CGP, we were able to obtain a learned distance function that is small, interpretable and thus easily portable. Throughout the different experiments we argue that the selected network properties have a positive impact on the learning process. Surprisingly, multiple distance functions evolved to encode sparse networks that, despite their sparsity, perform well across environments.

The main limitation of this work is the computational cost of the meta-evolution. Evolving a specific distance function for a specific set of problems is hardly tractable. This motivated the use of CGP with the hopes that the output could be reuse and generalize to unseen problems. As we demonstrate with the hopper and swimmer environments, the evolved distance functions do generalize to new tasks.

A direction for future work would be to define more neural network properties and carefully evaluate them to select those that are truly important for a network policy. This could guide the meta-evolution process more effectively and discover even more interesting and better performing distance functions. Furthermore, we used the same tasks and network architectures for all evaluated distance functions in this work. In future work, we aim to use curriculum learning to increase the complexity of tasks and the number of different architectures tested as the distance function improves over evolution.

This work demonstrates that evolution can be a powerful tool to learn new neural network representations. By using an interpretable form of evolution, genetic programming, we were able to study the evolved representation schemes and understand their functioning. Further study could give rise to

new, evolved types of neural networks, such as the sparse, compressed encoding discovered in this work.

REFERENCES

- [1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT press, 2016.
- [2] G. Cybenko, “Approximation by superpositions of a sigmoidal function,” *Mathematics of Control, Signals and Systems*, vol. 2, no. 4, pp. 303–314, Dec. 1989. [Online]. Available: <https://doi.org/10.1007/BF02551274>
- [3] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever, “Evolution strategies as a scalable alternative to reinforcement learning,” 2017. [Online]. Available: <http://arxiv.org/abs/1703.03864>
- [4] P. Templier, E. Rachelson, and D. G. Wilson, “A geometric encoding for neural network evolution,” in *Proceedings of the Genetic and Evolutionary Computation Conference*, ser. GECCO ’21. New York, NY, USA: Association for Computing Machinery, 2021, pp. 919–927. [Online]. Available: <https://doi.org/10.1145/3449639.3459361>
- [5] D. Wierstra, T. Schaul, J. Peters, and J. Schmidhuber, “Natural Evolution Strategies,” in *2008 IEEE Congress on Evolutionary Computation (IEEE World Congress on Computational Intelligence)*. Hong Kong, China: IEEE, Jun. 2008, pp. 3381–3387. [Online]. Available: <http://ieeexplore.ieee.org/document/4631255/>
- [6] J. F. Miller and S. Harding, “Cartesian Genetic Programming,” *Cartesian Genetic Programming*, p. 25, 2008.
- [7] K. O. Stanley and R. Miikkulainen, “Efficient evolution of neural network topologies,” in *Proceedings of the 2002 Congress on Evolutionary Computation. CEC’02 (Cat. No. 02TH8600)*, vol. 2. IEEE, 2002, pp. 1757–1762.
- [8] T. Elsken, J. H. Metzen, and F. Hutter, “Neural architecture search: A survey,” *Journal of Machine Learning Research*, vol. 20, no. 55, pp. 1–21, 2019. [Online]. Available: <http://jmlr.org/papers/v20/18-598.html>
- [9] M. Mitchell, *An introduction to genetic algorithms*. MIT press, 1998.
- [10] I. Rechenberg, “Evolutionstrategien,” in *Simulationsmethoden in der medizin und biologie*. Springer, 1978, pp. 83–114.
- [11] N. Hansen, “The CMA Evolution Strategy: A Tutorial,” *arXiv:1604.00772 [cs, stat]*, Apr. 2016, arXiv: 1604.00772. [Online]. Available: <http://arxiv.org/abs/1604.00772>
- [12] R. Ros and N. Hansen, “A simple modification in cma-es achieving linear time and space complexity,” in *International conference on parallel problem solving from nature*. Springer, 2008, pp. 296–305.
- [13] J. Schmidhuber, “Evolutionary principles in self-referential learning, or on learning how to learn: The meta-meta-... hook,” Ph.D. dissertation, Technische Universität München, 1987.
- [14] J. D. Co-Reyes, Y. Miao, D. Peng, E. Real, S. Levine, Q. V. Le, H. Lee, and A. Faust, “Evolving Reinforcement Learning Algorithms,” *arXiv:2101.03958 [cs]*, Jan. 2021, arXiv: 2101.03958. [Online]. Available: <http://arxiv.org/abs/2101.03958>
- [15] C. Lu, J. G. Kuba, A. Letcher, L. Metz, C. S. de Witt, and J. Foerster, “Discovered policy optimisation,” 2022.
- [16] M. T. Jackson, C. Lu, L. Kirsch, R. T. Lange, S. Whiteson, and J. N. Foerster, “Discovering temporally-aware reinforcement learning algorithms,” *The Twelfth International Conference on Learning Representations*, 2024.
- [17] J. Oh, M. Hessel, W. M. Czarnecki, Z. Xu, H. P. van Hasselt, S. Singh, and D. Silver, “Discovering reinforcement learning algorithms,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 1060–1070, 2020.
- [18] R. T. Lange, T. Schaul, Y. Chen, T. Zahavy, V. Dallibard, C. Lu, S. Singh, and S. Flennerhag, “Discovering Evolution Strategies via Meta-Black-Box Optimization,” Nov. 2022, arXiv:2211.11260 [cs]. [Online]. Available: <http://arxiv.org/abs/2211.11260>
- [19] R. Lange, T. Schaul, Y. Chen, C. Lu, T. Zahavy, V. Dalibard, and S. Flennerhag, “Discovering Attention-Based Genetic Algorithms via Meta-Black-Box Optimization,” in *Proceedings of the Genetic and Evolutionary Computation Conference*. Lisbon Portugal: ACM, Jul. 2023, pp. 929–937. [Online]. Available: <https://dl.acm.org/doi/10.1145/3583131.3590496>
- [20] D. G. Wilson, K. Harrington, S. Cussat-Blanc, and H. Luga, “Evolving Differentiable Gene Regulatory Networks,” *arXiv:1807.05948 [cs]*, Jul. 2018, arXiv: 1807.05948. [Online]. Available: <http://arxiv.org/abs/1807.05948>
- [21] B. Hanin and D. Rolnick, “How to start training: The effect of initialization and architecture,” *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [22] T. Hoefer, D. Alistarh, T. Ben-Nun, N. Dryden, and A. Peste, “Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks,” *The Journal of Machine Learning Research*, vol. 22, no. 1, pp. 10882–11005, 2021.
- [23] J. Frankle and M. Carbin, “The lottery ticket hypothesis: Finding sparse, trainable neural networks,” in *International Conference on Learning Representations*, 2018.
- [24] R. T. Lange and H. Sprekeler, “Lottery Tickets in Evolutionary Optimization: On Sparse Backpropagation-Free Trainability,” May 2023, arXiv:2306.00045 [cs]. [Online]. Available: <http://arxiv.org/abs/2306.00045>
- [25] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, “JAX: composable transformations of Python+NumPy programs,” 2018. [Online]. Available: <http://github.com/google/jax>
- [26] R. T. Lange, “evosax: Jax-based evolution strategies,” 2022. [Online]. Available: <http://github.com/RobertTLange/evosax>
- [27] C. D. Freeman, E. Frey, A. Raichuk, S. Girgin, I. Mordatch, and O. Bachem, “Brax - A differentiable physics engine for large scale rigid body simulation,” 2021. [Online]. Available: <http://github.com/google/brax>

APPENDIX

A. Implementation details

All the evaluations of the external and internal loops are carried out in parallel using GPU hardware acceleration with JAX [25] and open-source libraries such as Evosax [26] and Brax [27] that allow for the environment rollouts to run on the GPU. Research code will be made available on github. The search space of our CGP meta-evolution loop is defined by the number of nodes used, (64 here) to keep expressiveness, and by the size of the set of pre-defined $\mathcal{F}$ operators. Only 2 constants are used as possible input nodes to be selected by CGP, 0.1 and 1, as two additional input nodes. CGP operators are $+$, $-$, $\times$, $\div$, $>$, $<$, abs , $\exp$, $\sin$, $\cos$, $\log$, $\sqrt{\cdot}$.

B. Hyper-parameter table

CGP Hyperparameter	Value
Meta-population size	32
Number of generations	5000
Number of nodes	64
Replacement for NaN values	0
Number of used constants	2
number of used functions	12
Probability of mutating a function	0.15
Probability of mutating an input	0.15
Inner-loop Hyperparameter	Value
Inner-loop policy networks population size	32
GENE encoding dimension d	3
β	1/3
α	4 β
Activation functions used for the policies	$\tanh$
Strategy used for policy search	Sep CMA-ES
Policy network internal layer architecture	[128, 128]
Number of generations for HALFCHEETAH	500
Number of simulation steps for HALFCHEETAH	1000
Number of generations for WALKER2D	1500
Number of simulation steps for WALKER2D	1000